

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	S.MAGESH	Reg. No.:	192525002
Section / Batch:	B.TECH - AIML	Date:	23/07/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Q31. Debugging Selection Sort Code (Incorrect Outer Loop Range)

The following Selection Sort implementation skips the final required pass because of an incorrect outer loop boundary, which may leave the array partially unsorted for certain inputs. Carefully analyze the loop range and explain why the remaining unsorted portion is not handled correctly. Apply systematic debugging to correct the iteration structure and ensure complete sorting. Optimize the implementation by avoiding unnecessary swaps when the minimum element is already in the correct position. Analyze the time complexity of the corrected version and rewrite the final optimized code with proper comments.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n-2): # ERROR
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

A) UNDERSTANDING THE PROBLEM

The given Selection Sort program does not complete all the required passes because the outer loop runs only up to $n-2$. As a result, some elements may remain unsorted for certain inputs.

B) ANALYSIS

The outer loop should execute until $n-1$ so that every element is placed in its correct position. The given code also performs a swap even when the minimum element is already in the correct position, leading to unnecessary operations.

Time Complexity

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

C) CORRECTED CODE

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        min_idx = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j

        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]

    return arr
```

D) CONCLUSION

After correcting the outer loop and avoiding unnecessary swaps, the Selection Sort algorithm correctly sorts the entire array while maintaining the expected time complexity

Q32. Debugging Bubble Sort Code (Wrong Inner Loop Limit with Optimization Attempt)

The following Bubble Sort implementation attempts to reduce comparisons in each pass, but the inner loop limit is incorrect and causes some adjacent pairs to remain unchecked. Carefully inspect the loop boundary and explain why certain inversions survive after execution. Debug the algorithm step-by-step to restore correct sorting behavior. Optimize the final version using a swap flag so that unnecessary passes are avoided when the array becomes sorted early. Analyze the corrected algorithm in best, average, and worst cases and rewrite the final code with proper structure.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-2): # ERROR
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

A) UNDERSTANDING THE PROBLEM

The Bubble Sort program uses an incorrect inner loop boundary ($n-i-2$). Because of this, the last adjacent pair is not compared in every pass, leaving some elements unsorted.

B) ANALYSIS

The inner loop should run until $n-i-1$. A swap flag can also be used to stop the algorithm early if the array is already sorted.

Time Complexity

- Best Case: $O(n)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

C) CORRECTED CODE

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False

        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        if not swapped:
            break
```

```
return arr
```

D) CONCLUSION

The corrected Bubble Sort compares all required adjacent elements and stops early when the array becomes sorted, improving efficiency.

Q33. Debugging Linear Search Code (Incorrect Index Variable Update)

The following Linear Search implementation incorrectly updates the result index even after finding the target, causing the function to return the last occurrence instead of the first one. Carefully analyze the logic and identify why the intended search behavior is violated. Apply systematic debugging to ensure that the first valid occurrence is returned immediately when found. Optimize the implementation for readability and correct control flow. Analyze the time complexity of the corrected solution and rewrite the final code with clear comments.

```
def linear_search(arr, key):  
    pos = -1  
    for i in range(len(arr)):  
        if arr[i] == key:  
            pos = i  
    return pos
```

A) UNDERSTANDING THE PROBLEM

The given Linear Search continues updating the position even after finding the required element. Therefore, it returns the last occurrence instead of the first occurrence.

B) ANALYSIS

The function should immediately return the index once the key is found. This reduces unnecessary comparisons and produces the correct result.

Time Complexity

- Best Case: **$O(1)$**
- Average Case: **$O(n)$**
- Worst Case: **$O(n)$**

C) CORRECTED CODE

```
def linear_search(arr, key):  
  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return i  
  
    return -1
```

D) CONCLUSION

The corrected program returns the first occurrence of the required element and avoids unnecessary iterations.

Q34. Debugging Binary Search Code (Incorrect Mid Recalculation Logic)

The following Binary Search code recalculates the midpoint incorrectly because the boundary updates are inconsistent with the search direction. Carefully inspect the update rules for low and high and explain why valid regions may be discarded. Debug the code step-by-step so that the search interval shrinks correctly in all cases. Optimize the implementation for clarity and proper handling of edge cases. Analyze best, average, and worst-case time complexity and rewrite the corrected code with suitable documentation.

```
def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] > key:
            low = mid + 1 # ERROR
        else:
            high = mid - 1 # ERROR
    return -1
```

A) UNDERSTANDING THE PROBLEM

The Binary Search program updates the lower and upper boundaries incorrectly. This causes the algorithm to search the wrong half of the array and may fail to locate an existing element.

B) ANALYSIS

If the middle element is greater than the key, the upper boundary should be reduced. If it is smaller, the lower boundary should be increased.

Time Complexity

- Best Case: **$O(1)$**
- Average Case: **$O(\log n)$**
- Worst Case: **$O(\log n)$**
-

C) CORRECTED CODE

```
def binary_search(arr, key):
```

```
    low = 0
```

```
    high = len(arr) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if arr[mid] == key:
            return mid
```

```
        elif arr[mid] > key:
            high = mid - 1
```

```
else:  
    low = mid + 1
```

```
return -1
```

D) CONCLUSION

The corrected Binary Search reduces the search space correctly and efficiently finds the required element in a sorted array.

Q35. Debugging Closest Pair Code (Comparison with Wrong Variable)

The following brute force Closest Pair implementation calculates distances correctly but compares the newly computed distance against the wrong reference value. Carefully analyze the update logic and identify why the minimum pair is not always preserved. Apply debugging to ensure that the smallest distance found so far is maintained correctly throughout the nested loops. Optimize the implementation by avoiding repeated tuple indexing where possible. Analyze the time complexity of the corrected algorithm and rewrite the final version with proper comments.

```
import math  
def closest_pair(points):  
    min_dist = float('inf')  
    for i in range(len(points)):  
        for j in range(i+1, len(points)):  
            x1, y1 = points[i]  
            x2, y2 = points[j]  
            dist = math.sqrt((x1-x2)**2 + (y1-y2)**2)  
            if dist < i: # ERROR  
                min_dist = dist  
    return min_dist
```

A) Understanding the Problem

The program correctly calculates the Euclidean distance but compares the distance with the loop variable `i` instead of the current minimum distance. This prevents the algorithm from finding the actual closest pair.

B) Analysis

The comparison should be made with `min_dist`. Only when the newly calculated distance is smaller than the current minimum should the value be updated.

Time Complexity

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

C) Corrected Code

```
import math

def closest_pair(points):

    min_dist = float('inf')

    for i in range(len(points)):
        for j in range(i + 1, len(points)):

            x1, y1 = points[i]
            x2, y2 = points[j]

            dist = math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)

            if dist < min_dist:
                min_dist = dist

    return min_dist
```

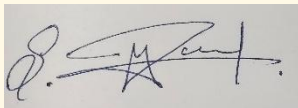
D) CONCLUSION

After comparing the calculated distance with min_dist, the algorithm correctly identifies the closest pair of points while maintaining the brute-force time complexity of $O(n^2)$.

Code of Conduct

*I **S.MAGESH(192525002)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student:



RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, wellstructured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____